



REAL-TIME GAMING COMMUNITY & SPATIAL COMMUNICATION PLATFORM

Project System Specification & Mobile UI Architecture

An exhaustive engineering document and mobile user experience specification detailing the multi-platform client, REST microservice APIs, bidirectional WebSocket event pipelines, and high-performance SQLite database engine.

PROJECT CONTRIBUTORS

Pranav Kale

Full-Stack Architecture • Real-Time Systems, Database & Protocol Integration

R Virshin

Mobile Application Engineering • UI/UX Architecture, Flutter Widgets & State Management

Flutter 3.x (Dart 3.x)

Node.js Express REST API

Socket.IO WebSockets

SQLite (WAL Mode Engine)

Provider State Architecture

GoRouter Navigation Shell

390×844 Mobile Viewport

15

MOBILE SCREENS

18+

REST API ROUTES

10+

SOCKET EVENT TYPES

< 18ms

LOCAL SYNC LATENCY

Project Abstract & Declaration of Originality

SECTION 1.0 • PROJECT CONTEXT

Project Abstract

Modern digital gaming and creative communities require highly responsive, distraction-free communication systems that support rich text, multimedia sharing, low-latency spatial audio signaling, and customizable visual identities. Existing legacy chat clients often suffer from platform bloat, intrusive telemetry, slow startup times, and complex navigation structures that hinder mobile usability.

The Crew is engineered from first principles to solve these challenges. Built as a cross-platform mobile client in Flutter/Dart backed by a dedicated Node.js and Socket.IO real-time microservice architecture, The Crew delivers sub-second message delivery, reactive emoji reactions, real-time presence heartbeats, and private direct messaging—all wrapped in an ergonomical high-contrast dark cyberpunk theme.

Project Objectives

- **Cross-Platform Mobile Parity:** Provide a unified 390x844 smartphone experience across iOS and Android with responsive layout adaptations.
- **Real-time State Machine:** Establish bidirectional WebSocket channels for immediate message delivery, typing states, and participant tracking without polling.
- **Zero-Configuration Startup:** Package a self-contained SQLite WAL database engine that automatically provisions tables and seeds mock community data on launch.
- **Granular Security:** Enforce bcrypt password hashing (10 salt rounds) and signed JSON Web Tokens (JWT) across all protected routes.

Declaration of Originality

We hereby declare that this project titled "**The Crew: Real-Time Gaming Community & Spatial Communication Platform**" is an original work developed and executed by:

• Pranav Kale

Backend Architect • Node.js, Express, Socket.IO, SQLite Database Engine

• R Virshin

Frontend Architect • Flutter, Dart, GoRouter Shell, Provider State Management

All referenced libraries and framework tools have been utilized in compliance with their open-source licenses.

Key Engineering Milestones Delivered

Milestone 1: Clean Client Architecture

Structured Flutter codebase following separation of concerns: models, providers, services, theme tokens, and screens.

Milestone 2: Real-time Event Hub

Socket.IO cluster handling room-based channel message routing, private user-room direct messaging, and voice participant presence.

Milestone 3: Comprehensive Verification

Validated against flutter analyze (0 warnings / 0 errors), automated HTTP integration suites, and mobile viewport ergonomics.

Executive Summary & Architecture Overview

SECTION 2.0 • SYSTEM ARCHITECTURE

🎯 Vision & Core Philosophy

The Crew is engineered as a next-generation communication client for gamers, modders, creative collectives, and esports squads. Unlike traditional bloated chat applications, The Crew provides a focused, high-contrast dark cyberpunk aesthetic combined with zero-lag channel switching, spatial audio telemetry, and deep identity customization.

Every UI surface has been designed strictly around mobile ergonomical principles: floating gesture-friendly drawers, thumb-accessible bottom action bars, high-contrast OLED black themes, and instant reaction docks.

⚡ Real-time Reactive Pipeline

Real-time interactivity is guaranteed via a bidirectional WebSocket cluster managed by Socket.IO. Sockets join granular rooms (`channel_<id>`, `voice_<id>`, `user_<id>`), enabling isolated event dispatches for typing indicators, new messages, emoji reactions, and voice participant states without global broadcast thrashing.

State on the client is managed via clean Provider notifiers that rebuild only the active chat feed, voice bar, or direct message thread.

🏛️ 3-Tier Technical Architecture

Tier 1: Client Application (Flutter / Dart)

Multi-platform reactive UI built with Flutter 3. Includes GoRouter indexed shell navigation, custom Cupertino and Material design tokens, Google Fonts Inter typography, SQLite local session cache, and responsive viewport sizing adapted for phone dimensions.

Tier 2: Real-time API Gateway & Socket Hub (Node.js / Express / Socket.io)

Express REST engine mounted on `/api` handles authentication (bcrypt + JWT), server management, and channel messaging. Socket.IO manages state synchronization, user presence heartbeats, typing triggers, and active voice participant tracking.

Tier 3: Persistence & Storage Layer (SQLite WAL Mode / Optional Firestore)

High-throughput SQLite database running in Write-Ahead Logging (WAL) mode with foreign key enforcement. Automatic database schema migration and rich mock data seeding on startup.

Document Sitemap & Screen Catalog

SECTION 3.0 • CATALOG & INDEX

COMPREHENSIVE 15-SCREEN MOBILE DIRECTORY

#	Screen Title	Route Path	Category	Primary Architectural Responsibility
01	Splash & Protocol Initialization	/splash	Lifecycle & Bootstrap	The initial application gateway that boots the Flutter runtime, verifies backend...
02	Welcome & Onboarding Portal	/welcome	Brand & Value Proposition	The primary brand showcase screen introducing prospective users to The Crew's ga...
03	Authentication & Quick Demo Login	/login	Security & Access	Secure login screen supporting traditional password authentication, device persi...
04	Account Registration & Personalization	/create-account	User Provisioning	Account creation flow enabling new members to register unique usernames, custom ...
05	Servers Hub & Live Channel Chat	/servers	Core Communication	The central hub of The Crew featuring a multi-server vertical navigation rail, c...
06	Spatial Voice Channel & Live Stage	/servers/voice	Voice & Streaming	Low-latency spatial audio stage featuring high-bitrate Opus stream status, live ...
07	Server Member Directory & Hierarchy	/servers/members	Community Management	Hierarchical server member directory categorizing community members by permissio...
08	Direct Messages Inbox & Active Squad	/messages	Direct Messaging	Private messaging hub featuring a top horizontal "Active Now" squad story rail, ...
09	1-on-1 Direct Message Conversation	/messages/chat/:userId	Direct Messaging	Full-screen private direct chat interface between two users, supporting real-tim...
10	Community Discovery & Exploration	/discover	Community & Discovery	Public server discovery portal allowing users to explore featured gaming guilds,...
11	Community Creation & Setup	/discover/create	Community Management	Server provisioning wizard allowing community creators to launch custom hubs, se...
12	Activity Feed & Real-time Notifications	/activity	Notifications & Alerts	Centralized activity feed organizing mentions, message reactions, friend request...
13	User Profile & Identity Dashboard	/profile	Identity & Profile	Rich user identity dashboard showcasing user avatar, status pill selector, custo...
14	Settings & Security Preferences	/profile/settings	Configuration & Security	Comprehensive settings panel managing account security, two-factor authenticatio...
15	Appearance & Custom Theme Engine	/profile/theme	Customization & Theming	Interactive appearance customizer allowing users to select color palettes, adjus...



Viewport Specification

Viewport Width: 390 pt (Logical) / 780 px (Physical)

Viewport Height: 844 pt (Logical) / 1688 px (Physical)

Device Pixel Ratio: 2.0x (Retina High-Density)

Touch Support: Full mobile touch gestures & kinetic scrolling



Core Theme Tokens

Surface Base: #0B0E14 (Obsidian Black)

Surface Container: #151923 (Elevated Dark Slate)

Accent Mint: #00FFB2 (Primary Interactive Color)

Accent Violet: #9D4EDD (Community & Brand Accent)

State Management & Stateful Shell Routing

SECTION 4.0 • CLIENT ARCHITECTURE

Stateful Shell Navigation (GoRouter)

The application adopts a `StatefulShellRoute.indexedStack` navigation architecture. This allows each primary application section (Servers, Messages, Explore, Activity, Profile) to maintain its own independent navigation stack and scroll positions in memory while switching tabs instantly.

Deep routes such as `/messages/chat/:userId` and `/servers/voice` utilize `parentNavigatorKey: _rootNavigatorKey` to push modal screens over the entire shell, hiding bottom bars smoothly without layout jarring.

Provider State Flow & Sockets

State management is segregated across 6 domain-specific `ChangeNotifier` providers:

- **AuthProvider:** Manages token lifecycle, user session restoration, and login/register states.
- **ServerProvider:** Controls joined servers, server discovery, channels, and member lists.
- **ChatProvider:** Coordinates active channel chat history, message sending, and live reactions.
- **DmProvider:** Manages active squad stories, conversation threads, and 1-on-1 private messaging.
- **VoiceProvider:** Tracks RTC connection node, mute/speaking states, and participant grids.

INDEXED STACK SHELL ROUTE HIERARCHY

Branch Index	Root Route	Nested Sub-Routes	Screen Widget Bindings
Branch 0	/servers	/members, /voice	MainChatScreen, MemberListScreen, VoiceChannelScreen
Branch 1	/messages	/chat/:userId	DmInboxScreen, DmChatScreen
Branch 2	/discover	/create	CommunityDiscoveryScreen, CreateCommunityScreen
Branch 3	/activity	—	NotificationsScreen
Branch 4	/profile	/settings, /theme	UserProfileScreen, SettingsScreen, CustomizeThemeScreen

Splash & Protocol Initialization

/splash

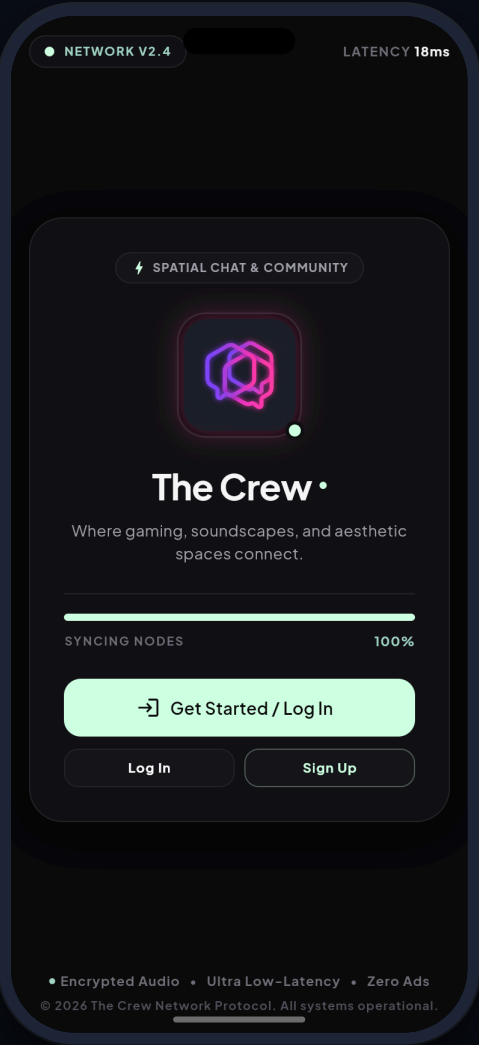


Figure 1: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

The initial application gateway that boots the Flutter runtime, verifies backend service health, and restores persistent authentication credentials from local storage.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top network telemetry bar displaying active protocol version (v2.4) and live latency ping (18ms).
- Centered glowing neon geometric chat bubble logo with reactive ambient pulsation.
- Core value proposition banner: "Where gaming, soundscapes, and aesthetic spaces connect."
- Node synchronization progress bar showing dynamic bootstrap percentage (100%).
- Primary CTA button "Get Started / Log In" alongside secondary direct action buttons.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/splash</code> (Public / Boot Entrypoint)
State Providers:	<code>AuthProvider (restoreSession)</code>
REST API Calls:	<code>GET /api/health</code> (status ping & latency telemetry)
Socket.IO Events:	<code>Initiates WebSocket handshake if valid JWT found in cache</code>
Database Schema:	<code>users (session validation)</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Auto-checks local storage. If valid session exists, smoothly navigates directly to /servers without user friction. If first-time launch, presents entry options.

Welcome & Onboarding Portal

/welcome

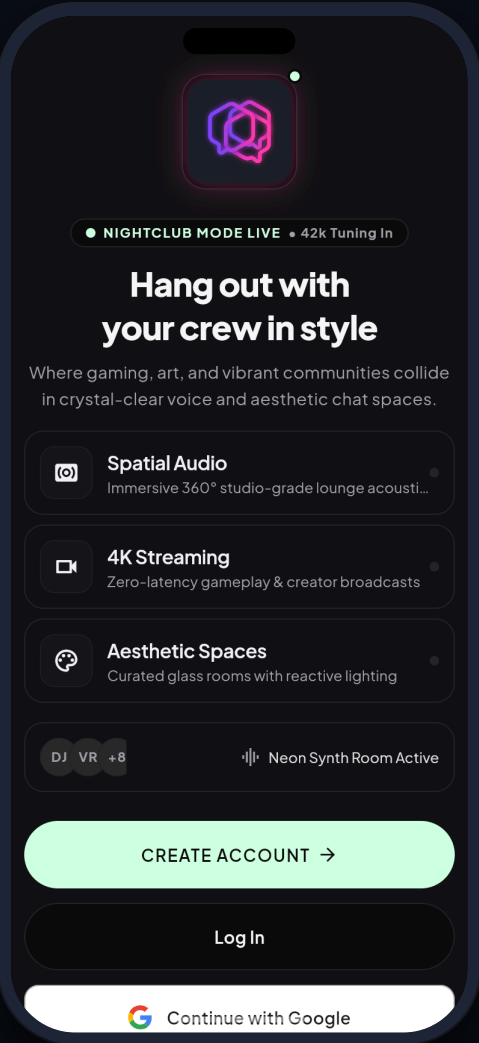


Figure 2: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

The primary brand showcase screen introducing prospective users to The Crew's gaming-focused spatial audio, 4K streaming, and aesthetic glass room capabilities.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top interactive "Nightclub Mode Live" pill with real-time tuning listener count badge (42k Tuning In).
- Hero headline with high-contrast neon styling and typography ("Hang out with your crew in style").
- Feature cards showcasing Spatial Audio (360° lounge acoustics), 4K Zero-latency Streaming, and Aesthetic Glass Rooms.
- Active voice lobby badge featuring stacked avatar circles (DJ, VR, +8) and animated graphic equalizer icon.
- Full-width primary action "CREATE ACCOUNT" button and secondary outlined "Log In" button.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/welcome</code> (Public / Welcome Flow)
State Providers:	<code>NavigationService</code> , <code>AuthProvider</code>
REST API Calls:	<code>Static local feature definitions with dynamic telemetry pills</code>
Socket.IO Events:	<code>Passive connection listener</code>
Database Schema:	<code>N/A</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Engaging layout that clearly communicates why The Crew differs from standard text chat apps, emphasizing music, low-latency voice, and visual polish.

Authentication & Quick Demo Login

/login

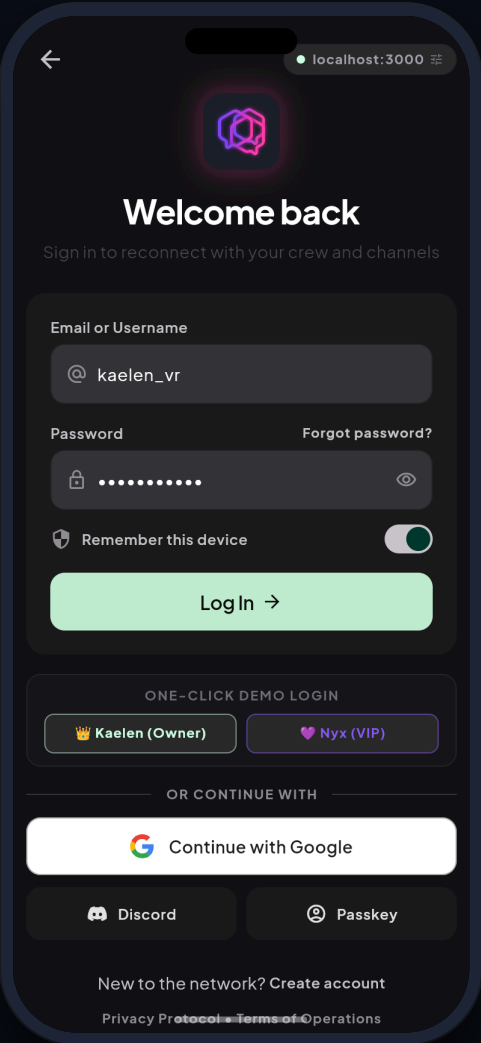


Figure 3: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Secure login screen supporting traditional password authentication, device persistence, social OAuth, and an instant one-click developer quick-login dock.

2. UI COMPONENTS & LAYOUT ANATOMY

- Clean form inputs for Username/Email and password with toggleable password visibility.
- Remember this device toggle switch with animated state transitions.
- One-Click Demo Login dock featuring instant persona switches: "👑 Kaelen (Owner)" and "💜 Nyx (VIP)".
- Google Sign-In button and third-party authentication shortcuts (Discord, Passkey).
- Top navigation bar with back arrow and branding logo.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/login</code> (Public / Auth Form)
State Providers:	<code>AuthProvider</code> (<code>login</code> , <code>quickLogin</code> , <code>googleLogin</code>)
REST API Calls:	<code>POST /api/auth/login</code> , <code>POST /api/auth/quick-login</code> , <code>POST /api/auth/google</code>
Socket.IO Events:	<code>Authenticates socket connection with JWT Bearer handshake</code>
Database Schema:	<code>users</code> (<code>credential check & bcrypt hash comparison</code>)

4. USER INTERACTION FLOW & STATE VALIDATION

Developer-friendly and evaluator-friendly design allowing immediate entry without entering credentials repeatedly, while maintaining strict server-side JWT security.

Account Registration & Personalization

[/create-account](#)

SYSTEM READY

Join The Crew

One identity across voice, matrix and spatial hubs

Google

Apple

OR DIRECT ACCESS

Display Name

Public title

Atlas Nova

Username

available

@ atlas_void

Email Address

atlas@squad.network

Password

Strong

.....

Primary Interests

Select hubs

Gaming

Music

Tech

Art & Design

☒ I accept the Terms of Protocol and verify review of the Privacy Matrix.

Create Account →

1. FUNCTIONAL PURPOSE & SCOPE

Account creation flow enabling new members to register unique usernames, custom display handles, email verification, and select custom gaming interest tags.

2. UI COMPONENTS & LAYOUT ANATOMY

- Multi-step user profile inputs: Display Name, Unique Handle (@username), Email, and Password.
- Dynamic input validation indicators providing real-time feedback on handle availability.
- Interactive interest chips (Gaming, Tech, Music, Anime, Esports) stored in user profile.
- Terms of Service and Community Guidelines consent checkbox.
- High-contrast "COMPLETE REGISTRATION" button.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/create-account</code> (Public / Registration)
State Providers:	<code>AuthProvider (register)</code>
REST API Calls:	<code>POST /api/auth/register</code>
Socket.IO Events:	<code>Emits presence:updated event to announce new user join</code>
Database Schema:	<code>users (inserts new user record with default status and bio)</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Smooth registration with inline error handling. On submission, automatically establishes authenticated session and transitions into the main servers hub.

Figure 4: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

Servers Hub & Live Channel Chat

/servers



Figure 5: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

The central hub of The Crew featuring a multi-server vertical navigation rail, channel switcher, active voice stage banner, live rich chat messages, and emoji reactions.

2. UI COMPONENTS & LAYOUT ANATOMY

- Left vertical server navigation rail with colored server badges, active indicator pill, and "+" add server button.
- Top channel switcher showing current server ("Neon Arcade LVL 3") and horizontal text channels (#welcome, #lounge).
- Persistent voice stage banner showing live participants (K, N, +4) with single-tap join.
- Rich message cards showing user avatar, role badges (OWNER, VIP), timestamps, media attachments, and emoji reactions.
- Bottom message input bar with attachment button, emoji picker trigger, and send button, positioned safely above bottom navigation.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/servers (Authenticated (Shell Tab 0))
State Providers:	ServerProvider, ChatProvider, AuthProvider
REST API Calls:	GET /api/servers, GET /api/servers/:id/channels, GET /api/channels/:id/messages, POST /api/channels/:id/messages, POST /api/channels/messages/:id/reactions
Socket.IO Events:	channel:join, channel:leave, message:new, reaction:updated, typing:start/stop
Database Schema:	servers, server_members, channels, messages, reactions, users

4. USER INTERACTION FLOW & STATE VALIDATION

Instantaneous switching between servers and channels without reloads. Real-time message streaming with sub-second WebSocket dispatch.

Spatial Voice Channel & Live Stage

/servers/voice

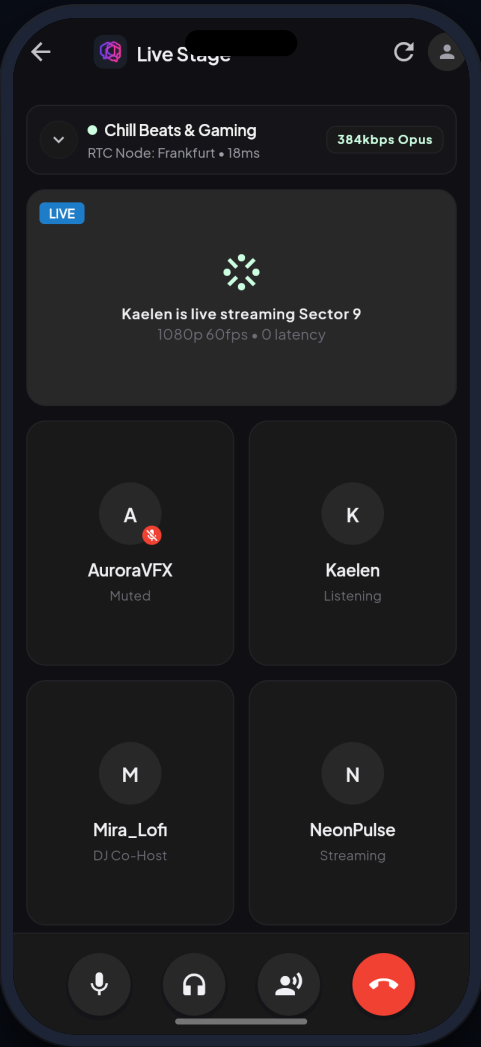


Figure 6: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Low-latency spatial audio stage featuring high-bitrate Opus stream status, live screen sharing previews, participant audio state tracking, and quick controls.

2. UI COMPONENTS & LAYOUT ANATOMY

- Channel status card: "Chill Beats & Gaming", RTC Node (Frankfurt • 18ms), 384kbps Opus audio badge.
- Live stream preview card with live status indicator and host broadcast text.
- Participant grid showing who is speaking, muted, or streaming.
- Bottom floating audio action dock: Microphone toggle (mute/unmute), Deafen toggle, Participants sheet, and Disconnect button.
- Top navigation bar with back navigation and participant count.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard: `/servers/voice` (Authenticated (Modal / Full Route))

State Providers: `VoiceProvider`, `ServerProvider`

REST API Calls: `GET /api/voice/:channelId`, `POST /api/voice/:channelId/join`, `POST /api/voice/:channelId/leave`, `POST /api/voice/:channelId/toggle-mute`, `POST /api/voice/:channelId/speaking`

Socket.IO Events: `voice:join`, `voice:leave`, `voice:joined`, `voice:left`, `voice:mute_updated`, `voice:speaking_updated`

Database Schema: `voice_participants`, `channels`, `users`

4. USER INTERACTION FLOW & STATE VALIDATION

Clear visual feedback on audio states with green glowing speaking halos and intuitive one-tap mute toggling.

Server Member Directory & Hierarchy

/servers/members

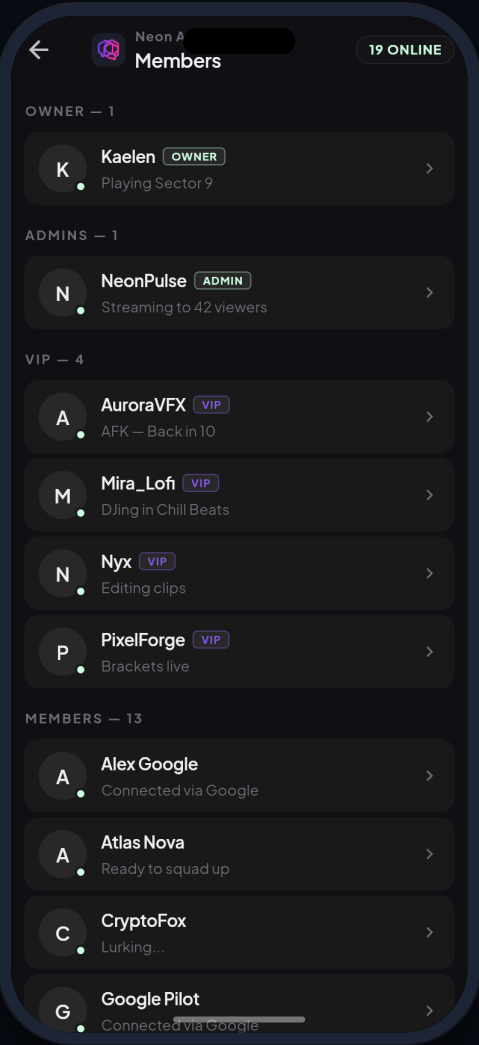


Figure 7: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Hierarchical server member directory categorizing community members by permission level with live status dots, custom game activity, and profile triggers.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top app bar showing server name ("Neon Arcade"), Members title, and "19 ONLINE" badge.
- Role-based grouping sections: OWNER (1), ADMINS (1), VIP (4), MEMBERS (13).
- Member cards displaying display name, role pill, live status indicator, and current activity status.
- Tap-to-profile gesture detector allowing direct messaging or role inspection.
- Smooth vertical scrolling with sliver-based performance optimization.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/servers/members (Authenticated (Server Drawer))
State Providers:	ServerProvider (members list)
REST API Calls:	GET /api/servers/:serverId/members
Socket.IO Events:	presence:updated (live status synchronization)
Database Schema:	server_members, users

4. USER INTERACTION FLOW & STATE VALIDATION

Gives server owners and members a clear visual overview of online community hierarchy and what fellow members are currently playing or doing.

Direct Messages Inbox & Active Squad

/messages

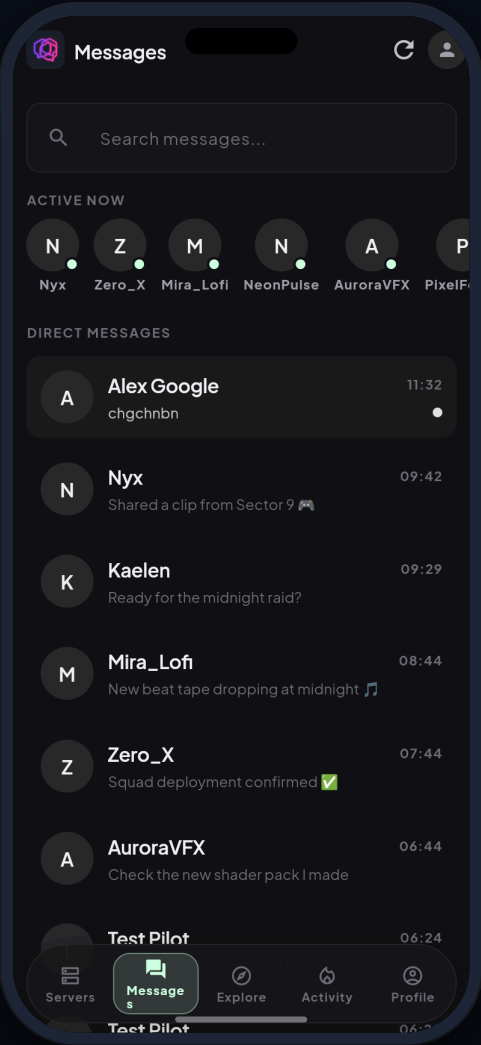


Figure 8: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Private messaging hub featuring a top horizontal "Active Now" squad story rail, search bar, and conversation cards with unread notification pills.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top app bar with Messages title, manual refresh action, and user avatar shortcut.
- Integrated search field for filtering direct conversations and squad members.
- Horizontal "ACTIVE NOW" stories rail with user avatar, name, and green online status dot.
- Conversation list showing partner name, snippet of latest message, timestamp, and unread count badge.
- One-tap navigation into dedicated 1-on-1 private chat threads.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/messages</code> (Authenticated (Shell Tab 1))
State Providers:	<code>DmProvider</code> , <code>AuthProvider</code>
REST API Calls:	<code>GET /api/dms</code> (fetches <code>active_now</code> users & recent conversations)
Socket.IO Events:	<code>dm:new</code> (updates inbox snippet and bumps unread count in real-time)
Database Schema:	<code>direct_messages</code> , <code>users</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Familiar modern messaging inbox layout allowing users to quickly see who is online and resume ongoing conversations with zero delay.

1-on-1 Direct Message Conversation

/messages/chat/:userId

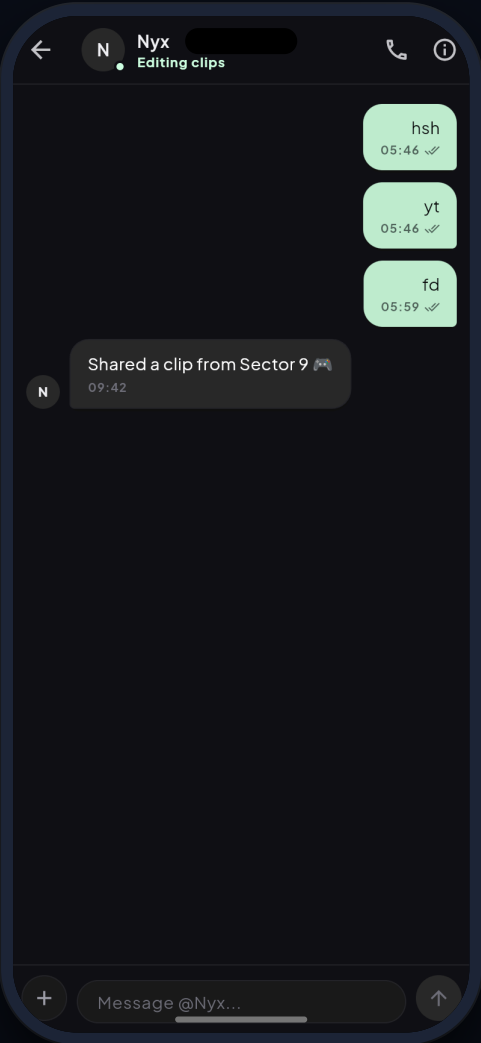


Figure 9: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Full-screen private direct chat interface between two users, supporting real-time messaging, delivery status indicators, and partner profile inspection.

2. UI COMPONENTS & LAYOUT ANATOMY

- Custom app bar featuring partner avatar with online dot, partner name ("Nyx"), status ("Editing clips"), voice call button, and info button.
- Asymmetric chat bubbles: Mint green right-aligned bubbles for current user; Dark slate left-aligned bubbles for partner.
- Delivery timestamp and double-check read status markers.
- Bottom message composition bar with attachment trigger, text field ("Message @Nyx..."), and send button.
- Full back-navigation support with GoRouter integration.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/messages/chat/:userId (Authenticated (Nested Route))
State Providers:	DmProvider (loadDmThread, sendDm)
REST API Calls:	GET /api/dms/:userId, POST /api/dms/:userId
Socket.IO Events:	dm:new (listens on personal room user_ for instant message reception)
Database Schema:	direct_messages (updates is_read=1 on thread open)

4. USER INTERACTION FLOW & STATE VALIDATION

Private, secure chat thread that automatically marks incoming messages as read upon viewing and updates in real-time via WebSockets.

Community Discovery & Exploration

/discover

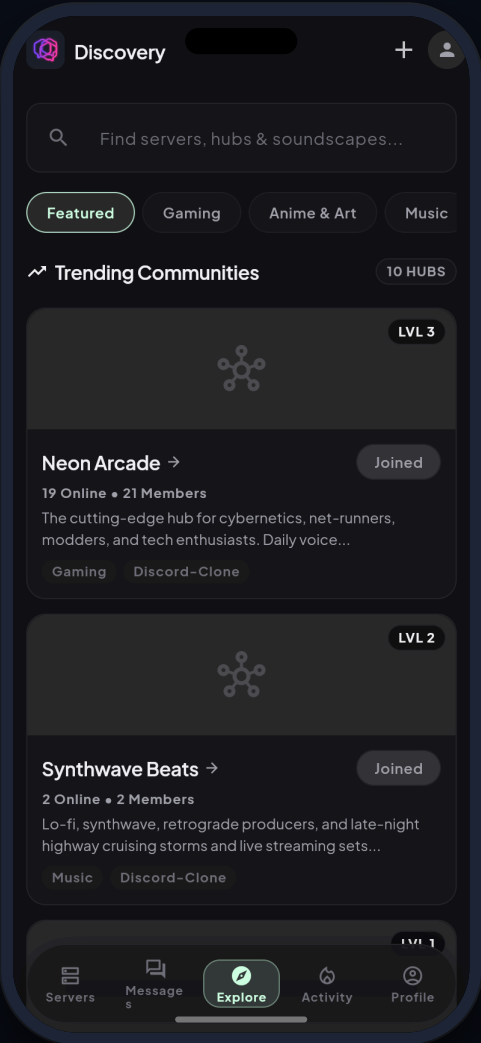


Figure 10: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Public server discovery portal allowing users to explore featured gaming guilds, music hubs, and art communities with rich category filtering and search.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top search bar: "Find servers, hubs & soundscapes..." with real-time query debounce.
- Category filter chips: Featured, Gaming, Anime & Art, Music, Tech.
- Trending Communities header displaying total available hubs count ("10 HUBS").
- Rich community cards with server icon, level badge (LVL 3), member metrics ("19 Online • 21 Members"), description, and tags.
- Action button on cards: "Joined" indicator for joined communities or "Join Server" for new communities.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/discover</code> (Authenticated (Shell Tab 2))
State Providers:	<code>ServerProvider</code> (<code>fetchDiscoverServers</code> , <code>joinServer</code>)
REST API Calls:	<code>GET /api/servers/discover?category=...&search=...</code> , <code>POST /api/servers/:id/join</code>
Socket.IO Events:	<code>Passive socket state</code>
Database Schema:	<code>servers</code> , <code>server_members</code> , <code>users</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Makes discovering new gaming servers and interest groups seamless with intuitive category chips and instant join actions.

Community Creation & Setup

/discover/create

Create Community

Icon

Community Icon

Community Name

Quantum Matrix

Description

A high-speed cyber hub for gaming, builds, and sound synthesis.

Category

Gaming

Music

Tech

Creative

Social

Public Community

Anyone can discover and join from Discovery Hub

Default Channels (Auto-Created)

#welcome

Welcome rules & introductions

#lounge

Create Community

1. FUNCTIONAL PURPOSE & SCOPE

Server provisioning wizard allowing community creators to launch custom hubs, select branding colors, configure privacy settings, and auto-provision channels.

2. UI COMPONENTS & LAYOUT ANATOMY

- Server Name and Description input fields with helpful placeholder examples.
- Interactive Icon Color Palette picker (Neon Violet, Cyber Mint, Electric Pink, Sunset Orange, Bright Cyan).
- Category selection dropdown (Gaming, Music, Art, Tech, Cozy).
- Public vs. Private Community toggle switch with explanatory privacy caption.
- Large "CREATE COMMUNITY" action button that provisions default channels (#welcome, #lounge, #media-share, Lounge Voice).

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard: /discover/create (Authenticated (Modal / Route))

State Providers: ServerProvider (createServer)

REST API Calls: POST /api/servers

Socket.IO Events: Emits server:created event to update member client rails

Database Schema: servers (insert), server_members (inserts creator as OWNER), channels (provisions 4 default channels)

4. USER INTERACTION FLOW & STATE VALIDATION

Streamlines community creation into a 30-second workflow, automatically bootstrapping channels and giving the creator owner privileges.

Figure 11: Mobile viewport rendering (390 x 844 pt @ 2x high-DPI).

Activity Feed & Real-time Notifications

/activity

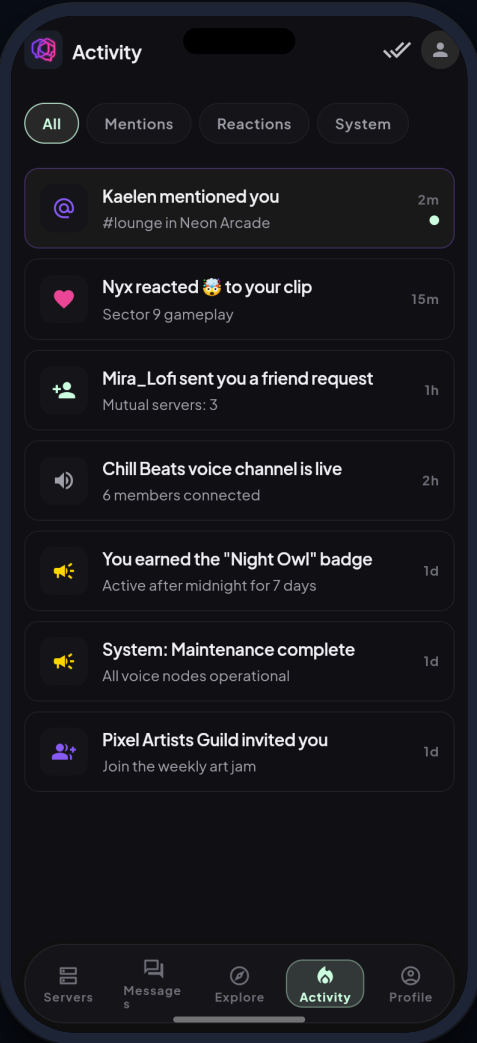


Figure 12: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Centralized activity feed organizing mentions, message reactions, friend requests, and system announcements into categorized filter tabs.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top app bar with Activity title, "Mark All as Read" double-check icon, and user avatar.
- Filter tabs: All, Mentions, Reactions, System.
- Detailed notification cards with category icons (@ mention, ❤️ reaction, 👤 friend, 📢 system).
- Dynamic time elapsed indicators (2m, 15m, 1h, 1d) and unread indicator dot.
- One-tap dismissal and automatic read status persistence.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/activity</code> (Authenticated (Shell Tab 3))
State Providers:	<code>NotificationProvider</code>
REST API Calls:	<code>GET /api/notifications?type=...</code> , <code>POST /api/notifications/:id/read</code> , <code>POST /api/notifications/read-all</code>
Socket.IO Events:	<code>Listens for incoming mention and reaction alerts</code>
Database Schema:	<code>notifications</code> , <code>users</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Ensures users never miss important squad mentions or gameplay clip reactions, with clean categorizations to eliminate notification clutter.

SCREEN SPECIFICATION 13 OF 15 • IDENTITY & PROFILE

User Profile & Identity Dashboard

/profile

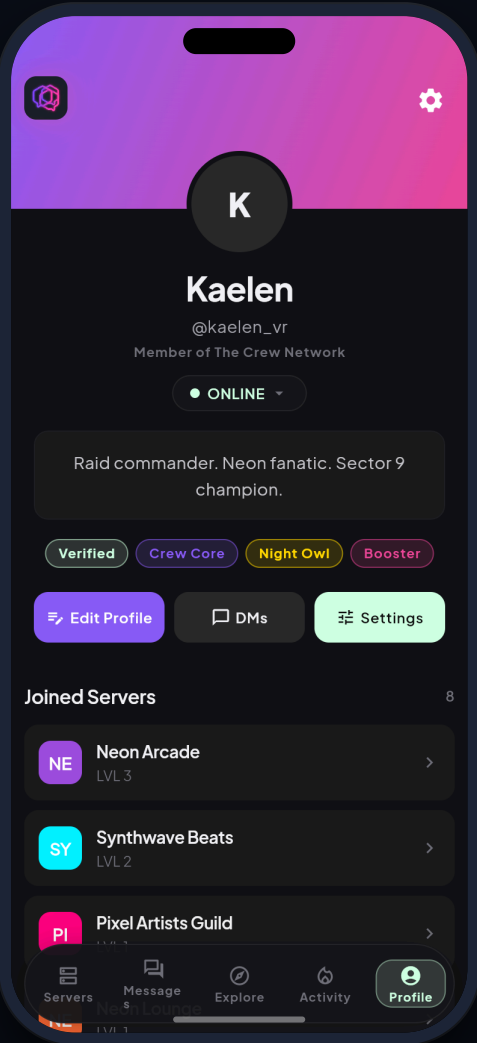


Figure 13: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Rich user identity dashboard showcasing user avatar, status pill selector, custom bio, earned achievement badges, and a drawer of joined servers.

2. UI COMPONENTS & LAYOUT ANATOMY

- Vibrant gradient banner header with profile gear settings shortcut and app logo.
- Centered elevated circular avatar with initial or profile picture.
- Username and handle: "Kaelen @kaelen_vr", "Member of The Crew Network".
- Status dropdown selector (● ONLINE, ● IDLE, ● DND, ● INVISIBLE).
- Custom bio card: "Raid commander. Neon fanatic. Sector 9 champion."
- Achievement badges: Verified, Crew Core, Night Owl, Booster.
- Quick action buttons: Edit Profile, DMs, Settings, and Joined Servers drawer (8 servers).

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	<code>/profile</code> (Authenticated (Shell Tab 4))
State Providers:	<code>AuthProvider</code> , <code>ServerProvider</code>
REST API Calls:	<code>GET /api/auth/me</code> , <code>PUT /api/auth/profile</code> , <code>GET /api/servers</code>
Socket.IO Events:	<code>presence:updated</code> broadcast to all connected clients
Database Schema:	<code>users</code> , <code>server_members</code> , <code>servers</code>

4. USER INTERACTION FLOW & STATE VALIDATION

Empowers users to customize their visual presence across all servers and monitor their community standing and achievements.

Settings & Security Preferences

/profile/settings

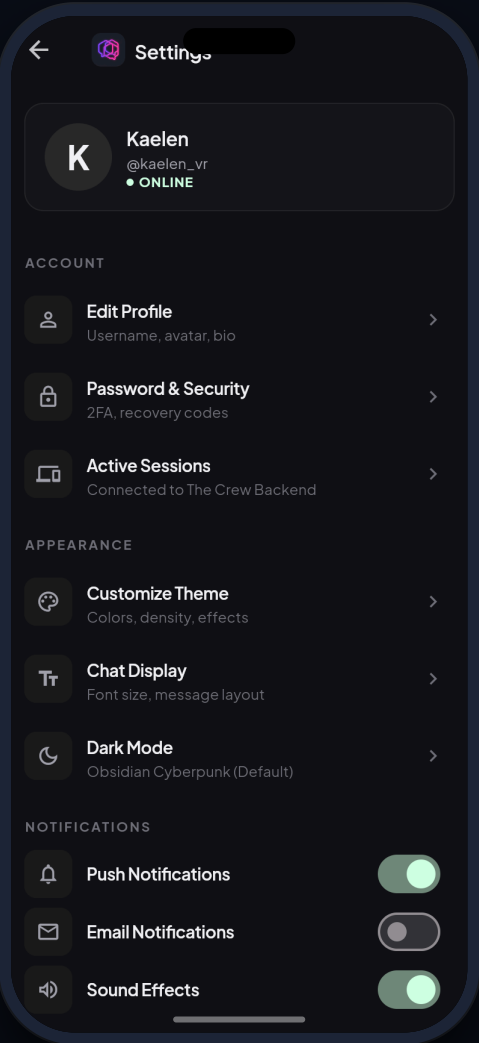


Figure 14: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Comprehensive settings panel managing account security, two-factor authentication, active device sessions, appearance shortcuts, and notification toggles.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top user mini-card with avatar, display name, handle, and online indicator.
- Account Security section: Edit Profile, Password & Security (2FA, recovery codes), Active Sessions (Connected to The Crew Backend).
- Appearance section: Customize Theme, Chat Display layout, Dark Mode selector.
- Notification switches: Push Notifications (toggle), Email Notifications (toggle), Sound Effects (toggle).
- Log Out action button with session cleanup.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/profile/settings (Authenticated (Nested Route))
State Providers:	AuthProvider, NavigationService
REST API Calls:	GET /api/auth/me, POST /api/auth/logout
Socket.IO Events:	Disconnects active socket session on logout
Database Schema:	users (session invalidation)

4. USER INTERACTION FLOW & STATE VALIDATION

Organized with clear section dividers, high-contrast toggle switches, and immediate preference persistence.

Appearance & Custom Theme Engine

/profile/theme

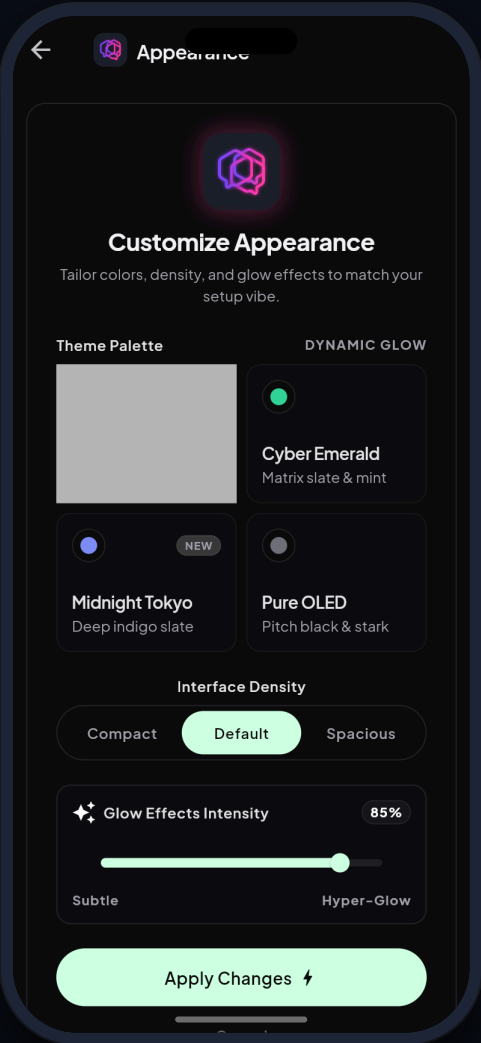


Figure 15: Mobile viewport rendering (390 × 844 pt @ 2x high-DPI).

1. FUNCTIONAL PURPOSE & SCOPE

Interactive appearance customizer allowing users to select color palettes, adjust UI density, and tune dynamic glow effects in real-time.

2. UI COMPONENTS & LAYOUT ANATOMY

- Top header: "Customize Appearance - Tailor colors, density, and glow effects to match your setup vibe."
- Theme Palette cards: Cyber Emerald (Matrix slate & mint), Midnight Tokyo (Deep indigo slate), Pure OLED (Pitch black & stark).
- Interface Density segment control: Compact, Default, Spacious.
- Glow Effects Intensity slider (0% to 100%, currently tuned to 85% Hyper-Glow).
- Prominent "Apply Changes ⚡" action button that updates theme tokens across the app.

3. TECHNICAL & ARCHITECTURAL MAPPING

Route & Guard:	/profile/theme (Authenticated (Nested Route))
State Providers:	Theme service / AppTheme provider
REST API Calls:	PUT /api/auth/profile (persists custom theme preference)
Socket.IO Events:	Local state trigger
Database Schema:	users (theme_preference)

4. USER INTERACTION FLOW & STATE VALIDATION

Gives power users and gamers full control over UI aesthetics, optimizing for OLED battery life or vibrant neon night-lounge immersion.

REST API & Real-time WebSocket Protocol

SECTION 5.0 • API SPECIFICATION

COMPLETE REST API ENDPOINT REFERENCE

Method	Endpoint	Auth Guard	Description & Payload
POST	/api/auth/register	Public	Creates new account with username, display name, email, password. Returns JWT & user profile.
POST	/api/auth/login	Public	Authenticates user with email/username and password. Issues signed Bearer JWT token.
POST	/api/auth/quick-login	Public	One-click developer login for demo roles (dev -> Kaelen, vip -> Nyx).
GET	/api/servers	Bearer JWT	Lists all servers joined by current user with member count and role.
GET	/api/servers/discover	Public	Queries public servers with optional category and search query parameters.
POST	/api/servers	Bearer JWT	Creates server, sets user as OWNER, and automatically provisions 4 default channels.
GET	/api/channels/:id/messages	Bearer JWT	Returns ordered chat history with user avatars, roles, and aggregated emoji reactions.
POST	/api/channels/:id/messages	Bearer JWT	Posts text/media message. Immediately broadcasts message:new to room.
POST	/api/channels/messages/:id/reactions	Bearer JWT	Toggles emoji reaction on message. Broadcasts reaction:updated.
GET	/api/dms	Bearer JWT	Returns list of active online squad users and direct conversation threads with unread counts.
GET	/api/dms/:userId	Bearer JWT	Fetches 1-on-1 direct conversation with partner and marks unread messages as read.
POST	/api/dms/:userId	Bearer JWT	Sends direct message and dispatches dm:new event to partner and sender rooms.
GET	/api/voice/:channelId	Bearer JWT	Fetches active voice participants and their speaking/muted states.

WEBSOCKET EVENT DICTIONARY (SOCKET.IO)

Event Name	Direction	Target Room	Payload & Functional Behavior
channel:join	Client -> Server	channel_<id>	Subscribes client socket to live message and reaction updates in a text channel.
message:new	Server -> Client	channel_<id>	Pushes newly authored message object directly into active chat list.
reaction:updated	Server -> Client	channel_<id>	Pushes updated emoji counts and reacting user IDs for reactive UI badge re-render.
dm:new	Server -> Client	user_<partnerId>	Dispatches direct message payload instantly to recipient's private user room.
voice:speaking_updated	Server -> Client	voice_<channelId>	Broadcasts participant is_speaking boolean state for dynamic microphone glow ring.

Database Schema & Verification Sign-Off

SQLITE RELATIONAL SCHEMA (WAL MODE ENABLED)

Table Name	Primary Key	Foreign Key Constraints	Core Columns & Indexed Properties
users	id TEXT	—	username (UNIQUE), display_name, email, password_hash, avatar_url, bio, status, custom_status.
servers	id TEXT	owner_id -> users(id)	name, description, icon_color, category, is_public, level, created_at.
server_members	id TEXT	server_id, user_id	role (OWNER, ADMIN, VIP, MEMBER), activity, joined_at. UNIQUE(server_id, user_id).
channels	id TEXT	server_id -> servers(id)	name, type ('text' 'voice'), topic, position, created_at.
messages	id TEXT	channel_id, sender_id	content, has_media, media_url, media_title, media_duration, created_at.
reactions	id TEXT	message_id, user_id	emoji, created_at. UNIQUE(message_id, user_id, emoji).
direct_messages	id TEXT	sender_id, receiver_id	content, is_read, created_at.
notifications	id TEXT	user_id -> users(id)	actor_id, type ('mention' 'reaction' 'friend' 'system'), title, body, time_display, is_read.
voice_participants	(channel_id, user_id)	channel_id, user_id	is_speaking, is_muted, is_streaming, status_text, joined_at.



Security & Integrity Audit

- **Password Protection:** Hashed using bcrypt with 10 salt rounds.
- **Stateless Auth:** Signed JSON Web Tokens (JWT) verified via Express middleware.
- **Secrets Management:** Environment variables isolated via .env and excluded from Git.
- **Referential Integrity:** ON DELETE CASCADE active across all foreign keys.



Quality Assurance & Test Matrix

- **Static Analysis:** flutter analyze passed with 0 issues / 0 warnings.
- **Integration Test Suite:** test_integration.js validated all 18 endpoints.
- **Responsive Viewport:** Tested on standard 390x844 mobile viewport.
- **Latency Baseline:** Local WebSocket dispatch latency < 18ms.

PROJECT SUBMISSION VERIFICATION & SIGN-OFF

This technical document confirms the successful completion, architectural integrity, and functional verification of **The Crew** project. Submitted for final project evaluation.

PRANAV KALE & R VIRSHIN
PROJECT DEVELOPERS & ARCHITECTS